KÜR · NICHT IM ABITUR · ERMITTLUNG K1

Eigene Datenbank bauen

Projektkarte K1 · CREATE TABLE, INSERT, Fremdschlüssel – und ein Ausblick

NAME _____ DATUM _____



> **FALL-EINSTIEG** *Juri ist begeistert: „Ich will die Fall-Datenbank selbst bauen, nicht nur benutzen!" Heute legst du Tabellen selbst an, füllst sie und erlebst, wie die Datenbank falsche Verweise abweist. Alles auf diesem Blatt ist **Kür – nicht im Abitur**.*

ERMITTLER-NOTIZ **KÜR – nicht im Abitur**

Im Abitur schreibst du **keine** CREATE- oder INSERT-Anweisungen, und den Verbund schreibst du ausschließlich mit `FROM … WHERE`. Die Schreibweisen mit JOIN auf diesem Blatt sind nur ein Ausblick (nicht im Abitur).

BEWEISSTÜCK 1 **Vom Schema zur Tabelle – die Asservatenkammer**

Die Detektei lagert Beweiskisten in Regalen. Aus jeder Relation wird mit `CREATE TABLE` eine Tabelle, aus jedem Attribut eine Spalte mit **Datentyp** (`INTEGER` = ganze Zahl, `TEXT` = Text). `PRIMARY KEY` kennzeichnet den Primärschlüssel, `FOREIGN KEY (…) REFERENCES …` einen Fremdschlüssel, `NOT NULL` verbietet leere Werte.

`Regal(RegalNr, Raum)`

`Kiste(KisteNr, Inhalt, ↑RegalNr)`

```
CREATE TABLE Regal (
  RegalNr INTEGER PRIMARY KEY,
  Raum    TEXT NOT NULL
);
CREATE TABLE Kiste (
  KisteNr INTEGER PRIMARY KEY,
  Inhalt  TEXT,
  RegalNr INTEGER,
  FOREIGN KEY (RegalNr) REFERENCES Regal(RegalNr)
);
```

Mit `INSERT INTO … VALUES (…)` fügst du Datensätze ein – die Werte in der Reihenfolge der Spalten, Texte in Hochkommata:

```
INSERT INTO Regal VALUES (1, 'Keller');
INSERT INTO Regal VALUES (2, 'Dachboden');
INSERT INTO Regal VALUES (3, 'Archivraum');
INSERT INTO Kiste VALUES (100, 'Fundbuch 2025', 1);
INSERT INTO Kiste VALUES (101, 'Kassenbons Stadtfest', 1);
INSERT INTO Kiste VALUES (102, 'Vereinsakten TuS', 2);
```

BEWEISSTÜCK 2 **Referentielle Integrität – die Datenbank passt auf**

Jeder Fremdschlüsselwert muss auf einen **existierenden** Primärschlüsselwert verweisen (oder NULL sein). Diese Kiste soll in Regal 7 – das es nicht gibt:

```
INSERT INTO Kiste VALUES (103, 'Stadtplan', 7);
```

Die Datenbank meldet `FOREIGN KEY constraint failed` und speichert nichts. (In SQLite muss die Prüfung mit `PRAGMA foreign_keys = ON` eingeschaltet sein – in der Sandbox ist sie es.)

BEWEISSTÜCK 3 **Ausblick: andere Schreibweisen für den Verbund (nicht im Abitur)**

Abitur-Stil – so schreibst du den Verbund in jeder Prüfung:

```
SELECT k.KisteNr, k.Inhalt, r.Raum
FROM Kiste k, Regal r
WHERE k.RegalNr = r.RegalNr
ORDER BY k.KisteNr;
```

KisteNr	Inhalt	Raum
100	Fundbuch 2025	Keller
101	Kassenbons Stadtfest	Keller
102	Vereinsakten TuS	Dachboden

Kür-Schreibweise mit JOIN … ON – die Verbundbedingung steht hinter ON, das Ergebnis ist gleich:

```
SELECT k.KisteNr, k.Inhalt, r.Raum
FROM Kiste k JOIN Regal r ON k.RegalNr = r.RegalNr
ORDER BY k.KisteNr;
```

LEFT JOIN behält alle Zeilen der linken Tabelle; fehlt der Partner, stehen rechts NULL-Werte. So findest du Regale **ohne** Kiste – das kann der Abitur-Verbund nicht:

```
SELECT r.RegalNr, r.Raum
FROM Regal r LEFT JOIN Kiste k ON r.RegalNr = k.RegalNr
WHERE k.KisteNr IS NULL;
```

RegalNr	Raum
3	Archivraum

Ermittlungsaufträge

NR.	STUFE	ORT	AUFTRAG
1	★☆☆	PAPIER	Die Detektei will ihre Zeuginnen und Zeugen speichern. Implementiere die `Tabelle` zu diesem Schema mit einer CREATE TABLE-Anweisung – ZeugeNr ist eine ganze Zahl, Name darf nie leer sein. `Zeuge(ZeugeNr, Name, Telefon)`
2	★☆☆	PAPIER	Trage die drei Zeugen mit `INSERT INTO` ein. Noa T. hat kein Telefon.
3	★★☆	PAPIER	Die Tabelle Vernehmung verweist auf Zeuge. Beurteile für jede Anweisung (nacheinander ausgeführt), ob die Datenbank sie annimmt oder abweist.

ZeugeNr	Name	Telefon
1	Mika R.	0000-4711
2	Lena K.	0000-0815
3	Noa T.	*NULL*

NR.	STUFE	ORT	AUFTRAG

```
CREATE TABLE Vernehmung (
  VernehmungNr INTEGER PRIMARY KEY,
  ZeugeNr      INTEGER,
  Datum        TEXT,
  FOREIGN KEY (ZeugeNr) REFERENCES Zeuge(ZeugeNr)
);
```

	Anweisung	angenommen? Warum?
a	INSERT INTO Vernehmung VALUES (1, 1, '2026-10-01');	
b	INSERT INTO Vernehmung VALUES (2, 4, '2026-10-02');	
c	INSERT INTO Vernehmung VALUES (1, 2, '2026-10-03');	
d	INSERT INTO Vernehmung VALUES (3, NULL, '2026-10-04');	
e	INSERT INTO Vernehmung VALUES (4, 3, '2026-10-05');	

4 ★★☆ **PAPIER**

Ausblick (nicht im Abitur): Verändere diese Abfrage im Abitur-Stil so, dass sie `JOIN … ON` verwendet, und gib das Ergebnis nach Auftrag 3 an.

```
SELECT z.Name, v.Datum
FROM Vernehmung v, Zeuge z
WHERE v.ZeugeNr = z.ZeugeNr
ORDER BY v.Datum;
```

.....

.....

.....

.....

5 ★★★ **PAPIER**

Ausblick (nicht im Abitur): Juri will wissen, welche Zeugen **noch nie** vernommen wurden. Entwirf eine Abfrage mit `LEFT JOIN` und erläutere, warum der Verbund im Abitur-Stil diese Frage nicht beantworten kann.

.....

.....

.....

.....

.....

6 ★★★ **LAB**

Teamprojekt: Plant eine eigene kleine Datenbank (Projektkarte auf der nächsten Seite) und setzt sie in der Sandbox um.

ABI-ECKE **Im Stil des Abiturs**

Die folgende Abfrage auf der Fall-Datenbank der Detektei nutzt eine Schreibweise, die im Abitur nicht vorkommt. **Entwerfen** Sie eine gleichwertige Abfrage im Stil der Ergänzenden Hinweise (Verbund über FROM und WHERE). [2 BE]

```
SELECT b.BeweisNr, b.Bezeichnung, o.Bezeichnung AS Fundort
FROM Beweisstueck b JOIN Ort o ON b.OrtNr = o.OrtNr
WHERE o.Stadtteil = 'Hafenviertel'
ORDER BY b.BeweisNr;
```

SPUR IM LAB **Ermittlung K1 „Eigene Datenbank bauen“**

Aufgaben: K1-E1-01 … K1-E1-06 (Kür, ohne Rangwirkung) · **Werkzeug:** Sandbox-Konsole im Schreibmodus
Jede Ausführung startet mit einer frischen Datenbank – kaputtmachen ist ausdrücklich erlaubt. Im Lab arbeitest du mit Ermittlern, Fällen und Personen.

COLD CASE · OHNE PUNKTE **Das Regal muss weg**

Was passiert, wenn du Regal 1 löschen willst, in dem noch Kisten stehen (`DELETE FROM Regal WHERE RegalNr = 1;`)? Probiere es in der Sandbox aus und bestimme, was `ON DELETE CASCADE` hinter `REFERENCES Regal(RegalNr)` ändern würde.

Projektkarte K1 · Unsere Datenbank

Team: _____ Thema: _____

Ideen: Schul-AG, Bandprobe, Tierheim, Sportturnier, Schulbibliothek, Fahrradwerkstatt …

Kernkriterium (wird im Lab geprüft)	erfüllt
mindestens **3 Tabellen**, jede mit Primärschlüssel	
mindestens **eine n:m-Beziehung**: Beziehungstabelle mit zusammengesetztem Primärschlüssel aus zwei Fremdschlüsseln	
jede Tabelle mit **mindestens zwei Datensätzen**	
drei Abfragen im Abitur-Stil (Verbund mit FROM … WHERE, GROUP-BY-Regel)	

1 · Schema in Kurzschreibweise (Primärschlüssel unterstrichen, ↑ vor Fremdschlüsseln)

2 · Das hat die Datenbank abgewiesen – Fehlermeldung und Erklärung

3 · Drei Abfragen im Abitur-Stil

Abfrage 1 – eine Tabelle, Bedingung im WHERE:

Abfrage 2 – Verbund über zwei Tabellen mit FROM … WHERE:

Abfrage 3 – über die n:m-Beziehung (drei Tabellen), gruppiert oder sortiert: